DASH-03: Executive Dashboards

> **Series:** DASH — Dashboard Design & Building | **Notebook:** 3 of 7 | **Created:** March 2026 | **Last Updated:** 10/14/2026

Overview

Executive dashboards distill complex observability data into a handful of business-meaningful KPIs. The goal is not to show everything — it is to show exactly what a decision-maker needs to assess system health in under 30 seconds. This notebook covers KPI selection, single-value tiles, trend lines, traffic-light patterns, MTTR calculations, and how to tell a story with data while avoiding information overload.

Table of Contents

1. KPI Selection Framework
2. Availability Percentage
3. Mean Time to Resolve (MTTR)
4. Problem Count and Trends
5. Service Health Score
6. Error Budget Tracking
7. Storytelling with Data
8. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	Dynatrace SaaS with Grail — the Dashboards app and DQL are not available on Dynatrace Managed, which keeps classic dashboards
Permissions	`storage:events:read`, `storage:metrics:read`, `storage:spans:read`
Data	detected problems with closed events (for MTTR), service spans
Prior Reading	DASH-01 and DASH-02

1. KPI Selection Framework

Selecting the right KPIs is the most important decision in executive dashboard design. Use these criteria:

The SMART Filter for Dashboard KPIs

Criterion	Question
Specific	Does this metric answer one clear question?
Measurable	Can Dynatrace calculate it reliably?
Actionable	Does a change in value trigger a specific response?
Relevant	Does the executive audience care about this?
Time-bound	Is the time range meaningful for decision-making?

Recommended Executive KPIs

KPI	Description	Tile Type	Threshold Example
Availability %	Uptime as a percentage of total time	Single value	Green: >99.9%, Yellow: >99.5%, Red: <99.5%

KPI	Description	Tile Type	Threshold Example
MTTR	Average time to resolve problems	Single value	Green: <1h, Yellow: <4h, Red: >4h
Active Problems	Current open problem count	Single value	Green: 0, Yellow: 1-3, Red: >3
Problem Trend	Problem count over 7-30 days	Line chart	Trending up = concern
Error Rate	Overall % of failed requests	Single value	Green: <1%, Yellow: <5%, Red: >5%

> **Tip:** Start with 4-5 KPIs. You can always add more later, but removing tiles executives have grown accustomed to is harder.

2. Availability Percentage

Availability is the most commonly requested executive metric. There are several ways to calculate it depending on what "availability" means in your organization.

Approach 1: Problem Impact — *not* an availability percentage

This is the tile most often asked for and most often wrong. Detected problems tell you **how much impact** occurred, not **what fraction of time** the estate was up.

> **Why a sum of problem durations is not downtime.** Problems overlap — Dynatrace opens one per
> affected entity, and many run concurrently. Measured on a live tenant over 7 days (08/25/2026):
> **319** AVAILABILITY problems across **254** entities summing to **714.7 hours** against a **168-hour** window — *4.3x the entire period*, with a peak of **73** problems open at once. Subtracting that from wall-clock time yields a negative availability, which is what this cell
> reported before it was corrected (**-325%**).

> The sum is a real and useful quantity — **entity-hours of impact** — but it is not elapsed
> downtime, and no filter makes it so. Elapsed downtime needs the *union* of the problem intervals,
> which this shape cannot express. For a real availability number use **Approach 2** below
> (success-rate, which measures requests rather than problems) or an **SLO** — see the SLO series,
> which exists for exactly this.

```
// Problem impact over 7 days. NOTE: this is entity-hours of impact, NOT downtime —
// problems overlap, so the sum exceeds wall-clock time. See the note above.
fetch dt.davis.problems, from:-7d
| filter event.status == "CLOSED"
| filter event.category == "AVAILABILITY"
| filter dt.davis.is_duplicate == false
| summarize
    impact_hours = sum(resolved_problem_duration / 1h),
    problem_count = count(),
    entity_arrays = collectArray(affected_entity_ids),
    longest_problem_hours = max(resolved_problem_duration / 1h)
// affected_entity_ids is an array: countDistinctExact on it counts distinct ARRAYS, not entities.
| fieldsAdd affected_entities = arraySize(arrayDistinct(arrayFlatten(entity_arrays)))
| fieldsRemove entity_arrays
```

Approach 2: Based on Service Success Rate

Calculate availability as the percentage of successful server-side requests.

> **Count the failures and subtract — never count the successes.** This is the one tile in the executive set where a plausible-looking query is catastrophically wrong, and the failure mode is a *credible number*, not an error:

Written as	Reads on a healthy estate
`countIf(otel.status_code != "ERROR")`	**0** — the field does not exist at all (no row in `dt.semantic_dictionary.fields`; the real field is `span.status_code`)

Written as	Reads on a healthy estate
`countIf(span.status_code != "ERROR")`	**0** — right field, wrong case. Grail values are lowercase `"error"`
`countIf(span.status_code != "error")`	**≈0** — right field, right case, wrong logic. `span.status_code` is **null on successful spans**, and `!=` against null yields null, not true
`total - countIf(span.status_code == "error")`	**99.598%** — the correct figure for the same 24 h window, tenant `yhu28601`, 08/11/2026

All three wrong forms put **0% availability** in front of an executive audience while the platform is running normally — the fastest possible way to destroy trust in the dashboard, and precisely the outcome the storytelling warning in §7 is about.

```
// Service-level availability based on span success rate
//
// Successes are DERIVED, not counted. span.status_code is null on successful
spans
// ("ok" is written vanishingly rarely), so countIf(span.status_code !=
"error")
// counts almost nothing and this tile would read ~0% availability on a
healthy
// estate — with no error to warn you. total - errors is the only correct
form.
fetch spans, from:-24h
| filter span.kind == "server"
| summarize {
    total = count(),
    errors = countIf(span.status_code == "error")
  }
| fieldsAdd successes = total - errors
| fieldsAdd availability_pct = round(100.0 * successes / total, decimals: 3)
```

3. Mean Time to Resolve (MTTR)

MTTR here is the average duration of closed problems. A problem closes when its events close, so this measures how long anomalies lasted — including issues that recovered on

their own — not how long a team took to resolve them, and it is not DORA's failed-deployment recovery time (ADOPT-03 § 3). Label the tile accordingly.

Current MTTR (Single Value)

```
// Average MTTR over last 7 days in hours
fetch dt.davis.problems, from:-7d
| filter event.status == "CLOSED"
| filter dt.davis.is_duplicate == false
| summarize mttr_hours = avg(resolved_problem_duration / 1h)
```

MTTR Trend Over Time

Show MTTR as a daily trend to highlight improvement or degradation.

```
// MTTR trend — daily average over 30 days
fetch dt.davis.problems, from:-30d
| filter event.status == "CLOSED"
| filter dt.davis.is_duplicate == false
| makeTimeseries mttr_hours = avg(resolved_problem_duration / 1h),
interval:24h, time:event.end
```

4. Problem Count and Trends

Problem trends reveal whether operational health is improving or declining over time.

Weekly Problem Trend by Category

```
// Problem trend by category over 7 days
// time:event.start bins each problem on the day it opened, not the day it was
last updated
fetch dt.davis.problems, from:-7d
| filter dt.davis.is_duplicate == false
| makeTimeseries problem_count = count(), interval:24h, by:{event.category},
time:event.start
```

Problems by Category — Executive Summary Table

```
// Problem summary by category — executive table tile
fetch dt.davis.problems, from:-7d
| filter dt.davis.is_duplicate == false
| summarize {total = count(), active = countIf(event.status == "ACTIVE"),
closed = countIf(event.status == "CLOSED")}, by:{event.category}
| sort total desc
```

5. Service Health Score

A composite health score combines multiple signals into a single number. This is useful for executives who want one metric per service.

Simple Health Score Formula

Health = 100 - (error_rate_penalty + latency_penalty)

Factor	Penalty Calculation
Error rate > 1%	+20 penalty
Error rate > 5%	+50 penalty
P95 latency > 1s	+15 penalty
P95 latency > 3s	+35 penalty

```
// Service health score — composite metric for executive dashboard
fetch spans, from:-1h
| filter span.kind == "server"
| summarize {total = count(), errors = countIf(span.status_code == "error"),
p95_ns = percentile(duration, 95)}, by:{dt.entity.service}
| fieldsAdd error_rate = 100.0 * errors / total
| fieldsAdd p95_ms = p95_ns / 1ms
| fieldsAdd error_penalty = if(error_rate &gt; 5, then: 50, else:
if(error_rate &gt; 1, then: 20, else: 0))
| fieldsAdd latency_penalty = if(p95_ms &gt; 3000, then: 35, else: if(p95_ms
&gt; 1000, then: 15, else: 0))
| fieldsAdd health_score = 100 - error_penalty - latency_penalty
| fieldsKeep dt.entity.service, health_score, error_rate, p95_ms
| sort health_score asc
| limit 10
```

6. Error Budget Tracking

Error budgets quantify how much failure is acceptable before an SLA breach. For a 99.9% SLA, the monthly error budget is 43.2 minutes of downtime.

SLA Target	Monthly Error Budget
99.99%	4.32 minutes
99.9%	43.2 minutes
99.5%	3.6 hours
99.0%	7.2 hours

> **An error budget cannot be computed from summed problem durations.** The 43.2-minute figure is
> *wall-clock* downtime, while summing problem durations counts overlapping problems repeatedly. On the
> verification tenant that produced a budget-remaining of **-6,824,364%** — 2,948,168 minutes of
> "downtime" claimed inside a 30-day window. Error budgets are an SLO construct: define the SLO, and the
> burn rate follows from it. See **SLO-03** (composition and error budgets) and **SLO-04**

> (burn-rate
> alerting). The query below reports the impact figures this data *can* support.

Problem Impact Against the Budget Reference

```
// Problem impact over 30 days, against the 99.9% budget reference.
// impact_minutes is entity-minutes of impact, NOT wall-clock downtime — it is
shown
// beside the budget for scale, not subtracted from it. Use an SLO for a real
budget.
fetch dt.davis.problems, from:-30d
| filter event.status == "CLOSED"
| filter event.category == "AVAILABILITY"
| filter dt.davis.is_duplicate == false
| summarize
    impact_minutes = sum(resolved_problem_duration / 1m),
    problem_count = count(),
    entity_arrays = collectArray(affected_entity_ids)
| fieldsAdd budget_reference_min_999 = 43.2
// affected_entity_ids is an array: countDistinctExact on it counts distinct
ARRAYS, not entities.
| fieldsAdd affected_entities =
arraySize(arrayDistinct(arrayFlatten(entity_arrays)))
| fieldsRemove entity_arrays
```

7. Storytelling with Data

An executive dashboard should tell a story without explanation. Follow these storytelling principles:

Layout Pattern for Executive Dashboard

Row	Left	Center	Right
Top	Availability % (single value)	MTTR (single value)	Active Problems (single value)
Middle	Problem trend (7d line chart)	—	Error budget remaining (single value)

Row	Left	Center	Right
Bottom	Service health table (top 5 worst)	—	—

Storytelling Principles

1. **Lead with the headline** — the top row answers "are we healthy?" in 3 numbers
2. **Support with trends** — the middle row shows direction (improving or declining)
3. **Provide detail on demand** — the bottom row offers specifics for those who want them
4. **Use color intentionally** — green means healthy, yellow means watch, red means act
5. **Avoid decoration** — no gratuitous pie charts, no unnecessary 3D effects, no logos

> **Warning:** Executives will lose trust in a dashboard that shows misleading data. Always validate that your availability and MTTR calculations match what the team reports manually. A discrepancy between the dashboard and a status meeting destroys credibility.

8. Summary and Next Steps

In this notebook you learned:

- How to select KPIs using the SMART filter
- Two approaches to calculating availability percentage
- MTTR calculation and trending over time
- Problem count analysis by category and status
- Composite service health scores
- Error budget tracking for SLA targets
- Storytelling principles for executive dashboard layout

Next: In **DASH-04: Operations Dashboards**, we move to the operations tier — real-time service monitoring, infrastructure health, log volume tracking, and problem/alert integration.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.